

LLM-Tor: Unlinkable Access to Commercial LLM APIs via Blind Signatures and Tor

Prince Gupta

March 2026

Abstract

LLM-Tor is a cryptographically enforced privacy layer that enables unlinkable access to commercial Large Language Model (LLM) APIs. It separates payment identity from model usage using RSA blind signatures (RFC 9474) and onion-routed communication via Tor. The system prevents correlation between user identity and chat content at the proxy layer while maintaining compatibility with frontier commercial LLM providers.

Contents

1	Introduction	3
2	Background	3
2.1	Blind Signatures	3
2.2	Tor Onion Routing	3
3	System Architecture	4
3.1	Components	4
3.2	Protocol Phases	4
3.2.1	Phase 1: Credit Purchase	4
3.2.2	Phase 2: Blind Token Issuance	4
3.2.3	Phase 3: Anonymous Redemption	4
4	Token Structure	5
5	Abuse Prevention System	5
5.1	Motivation	5
5.2	Dual-Token Design	5
5.3	Issuance	5
5.4	Blacklisting Mechanism	6
5.5	Anonymity Properties	6
5.6	Client-Side Backup and Recovery	6
6	Security Properties	6
6.1	Unlinkability	6
6.2	Single-Use Enforcement	7
6.3	Identity Separation	7
7	Moderation Requirement	7
8	Threat Model	7

8.1	Adversaries	7
8.2	Security Goals	7
8.3	Out of Scope	7
9	Key Rotation Policy	7
10	Limitations	8
11	Future Work	8
12	Conclusion	8

1 Introduction

Commercial LLM APIs require authenticated access tied to user accounts. This creates a structural privacy issue: prompts and responses become permanently associated with identifiable users.

LLM-Tor introduces a cryptographic separation between:

- Payment identity
- Usage authorization
- Model inference requests

The goal is to ensure that even the proxy operator cannot link a specific prompt to a specific paying customer.

2 Background

2.1 Blind Signatures

LLM-Tor uses RSA Blind Signatures as defined in RFC 9474, specifically the `RSABSSA-SHA384-PSS-Randomized` suite.

A blind signature scheme allows a signer to sign a message without learning its contents. The protocol consists of:

1. Blinding
2. Signing
3. Unblinding
4. Verification

Security relies on the hardness of RSA and the one-more RSA assumption.

Let m be a random token. The client computes a blinding factor r and sends:

$$m' = m \cdot r^e \pmod{N}$$

The server blind-signs:

$$s' = (m')^d \pmod{N}$$

The client unblinds:

$$s = s' \cdot r^{-1} \pmod{N}$$

The resulting signature s is valid for m , but the signer never sees m .

2.2 Tor Onion Routing

Tor provides layered encryption and multi-hop routing to hide client IP addresses. LLM-Tor routes all anonymous redemption traffic through Tor. In future, the inference endpoint will be exclusively accessible as a Tor onion service.

Reasons for using Tor:

- Prevent IP-level linkage during token redemption
- Eliminate DNS metadata leakage
- Provide probabilistic anonymity against network observers

LLM-Tor does not rely on Tor for cryptographic unlinkability; Tor and blind signatures are independent, complementary protections.

3 System Architecture

3.1 Components

- Payment Service
- Credit Ledger
- Blind Signing Service
- Token Redemption Proxy
- Moderation Layer
- Upstream LLM Provider

3.2 Protocol Phases

3.2.1 Phase 1: Credit Purchase

1. User purchases credits via standard payment provider.
2. Server records credit balance linked to user identity.

Identity exists in this phase.

3.2.2 Phase 2: Blind Token Issuance

1. Client generates random token T .
2. Client blinds T using model-specific public key.
3. Server verifies credit availability.
4. Server blind-signs the blinded token.
5. Client unblinds signature.

The server never observes T in unblinded form. Identity exists in this phase.

3.2.3 Phase 3: Anonymous Redemption

1. Client establishes Tor circuit.
2. Client submits (T, σ) .
3. Server verifies RSA signature.
4. Server checks token not previously spent.
5. Server forwards prompt to upstream LLM.

No identity is present in this phase.

4 Token Structure

Each token contains:

- A random 128-bit value.
- Model identifier: implicitly linked by choosing the public key specific to that model.
- Key identifier (for rotation) [planned].

Spent tokens are stored as $\text{hash}(T)$ to prevent replay.

5 Abuse Prevention System

5.1 Motivation

Per-request LLM tokens are single-use and fully unlinkable; there is no stable identifier that the proxy can use to rate-limit or ban a misbehaving user. Content moderation rejects individual bad requests, but does not prevent a determined adversary from repeatedly issuing new tokens and continuing to submit harmful content. The abuse-token scheme provides the proxy with a *stable but anonymous* handle per user. This is largely motivated from legal compliance and ability to adhere to upstream TOS.

5.2 Dual-Token Design

Two tokens serve complementary roles:

Token	Structure	Validity
Permanent (A_p)	48 random bytes	Lifetime of account
Transient (A_t)	44 random bytes 4-byte epoch	One calendar month

Permanent token A_p . Issued once per account. Remains valid indefinitely. Blacklisting A_p permanently bans the anonymous pseudonym.

Transient token A_t . Issued once per calendar month. Blacklisting A_t provides a time-bounded ban; the permanent token continues to provide a long-lived identifier.

The epoch is defined as $\text{epoch} = \text{year} \times 12 + \text{month}$ (month 1-indexed), encoded as a big-endian 32-bit unsigned integer appended to the random bytes. Each token type uses a dedicated RSA key pair, independent of the per-model keys.

Both tokens are blind-signed, so the server cannot correlate issuance (where the user is authenticated) with redemption (where the user is anonymous over Tor).

5.3 Issuance

Issuance follows the same blind-signature protocol as per-request tokens, using two dedicated RSA key pairs:

1. Client generates random bytes (48 for permanent; 44 random || 4-byte epoch for transient).
2. Client blinds with the corresponding abuse public key and sends the blinded token to the server over an authenticated session.
3. Server checks that the token type has not already been issued for this account in the current epoch:

- Permanent: rejects if `PermanentAbuseTokenIssuedAt` is non-nil.
 - Transient: rejects if `TransientAbuseTokenEpoch` equals the current epoch.
4. Server blind-signs and records the issuance flag atomically (protected by a per-user distributed semaphore to prevent concurrent double-issuance).
 5. Client unblinds and verifies the signature.

5.4 Blacklisting Mechanism

Blacklisted tokens are stored by their hash:

$$\text{DocID} = \text{base64}(\text{SHA-256}(A))$$

This mirrors the pattern used for spent per-request tokens. The server checks both the permanent and transient blacklists on every anonymous request before forwarding to the upstream LLM.

5.5 Anonymity Properties

Unlinkability. Because both A_p and A_t are blind-signed, the server cannot link the issuance event (authenticated session) to any specific anonymous request. The permanent token creates a persistent *anonymous pseudonym*: all requests bearing the same A_p are linkable to each other, but not to the paying identity.

Pseudonymity trade-off. The scheme intentionally trades full per-request unlinkability for pseudonymity within a session-less context. This is a deliberate design decision: without any stable identifier the proxy has no mechanism to enforce abuse bans. The permanent token carries no real-world identity; it is simply a stable random value whose signature the proxy can verify.

Tor independence. Abuse-token verification occurs at the application layer. Tor provides network-layer anonymity independently; the two protections are orthogonal.

5.6 Client-Side Backup and Recovery

The client encrypts the stored tokens for backup using:

- Key derivation: PBKDF2 (SHA-256, 600 000 iterations, 128-bit random salt).
- Encryption: AES-256-GCM with a 96-bit random IV.
- Format: `base64(salt || IV || ciphertext)`.

The encrypted backup is always exported as a local file (`.llmtorbak`); server-side backup storage is opt-in. In the server-sync case the server stores only the client-encrypted ciphertext and cannot decrypt it.

6 Security Properties

6.1 Unlinkability

Under the security assumptions of RSA blind signatures, issued tokens cannot be linked to redeemed tokens.

6.2 Single-Use Enforcement

Tokens are marked spent atomically upon first successful verification. Replay attempts are rejected.

6.3 Identity Separation

Payment identity never appears in redemption requests. Redemption occurs exclusively over Tor.

7 Moderation Requirement

LLM-Tor performs content moderation prior to forwarding requests upstream.

Reasons:

- Legal compliance in certain jurisdictions
- Abuse mitigation (spam, illegal content)
- Prevention of infrastructure misuse

Moderation is performed at the point where content arrives at the proxy; at that point the content is not attributable to any user identity.

8 Threat Model

8.1 Adversaries

- Honest-but-curious proxy operator
- External network observer
- Malicious user attempting double-spend
- Upstream LLM provider

8.2 Security Goals

- Prevent proxy from linking prompt to payment identity
- Prevent token double spending
- Prevent IP-level identity leakage

8.3 Out of Scope

- Global passive adversary controlling Tor
- Endpoint compromise
- Logging by upstream LLM providers
- Stylometric fingerprinting

9 Key Rotation Policy

TODO

10 Limitations

LLM-Tor does not provide absolute anonymity. It provides unlinkability at the proxy layer under standard cryptographic assumptions, with the added benefit of network anonymity via Tor built into the official client.

11 Future Work

- Exclusive Tor onion hosting for the proxy microservice
- Key rotation
- Zero-knowledge token systems
- Anonymous payment integration
- Post-quantum blind signatures
- Decentralized moderation

12 Conclusion

LLM-Tor demonstrates that cryptographic blind signatures combined with onion routing can provide practical unlinkability for commercial LLM API access, without requiring local model hosting.